

Design patterns

Creational

Abstract Factory – Usage

The Abstract Factory pattern provides an interface for creating **families of related objects** without specifying their concrete classes.

It ensures that products created together are **compatible**, **consistent**, and **interchangeable** as a group.

This pattern is often called the “**factory of factories.**”

When to Use Abstract Factory

You need to create related objects that must match

Example: A GUI system where all widgets (Button, Checkbox, Window) must follow the same theme (Dark, Light, Modern).

You want to enforce consistency across product families

Switching factories switches the entire family of components automatically.

You want to isolate client code from concrete classes

Clients only interact with interfaces — never knowing which concrete classes are used.

You expect the system to grow with new families

You can add new factories (themes/platforms/products) without touching existing code.

Example:

```
public interface AbstractFactory {  
    ProductA createProductA();  
    ProductB createProductB();  
}
```

```
public class ConcreteFactory_Black implements AbstractFactory {  
    @Override  
    public ProductA createProductA() {  
        return new ConcreteProductA_Black();  
    }  
    @Override  
    public ProductB createProductB() {  
        return new ConcreteProductB_Black();  
    }  
}
```

```
-----  
public class ConcreteFactory_White implements AbstractFactory {  
    @Override  
    public ProductA createProductA() {  
        return new ConcreteProductA_White();  
    }  
    @Override  
    public ProductB createProductB() {  
        return new ConcreteProductB_White();  
    }  
}
```

```
-----  
public interface ProductA {  
    void use();  
};
```

```
-----  
public interface ProductB {  
    void use();  
};
```

```
-----  
public class ConcreteProductA_Black implements ProductA {  
    @Override  
    public void use() {  
    }  
}
```

```
-----  
public class ConcreteProductA_White implements ProductA {  
    @Override  
    public void use() {  
    }  
}
```

```
-----  
public class ConcreteProductB_Black implements ProductB {  
    @Override  
    public void use() {  
    }  
}
```

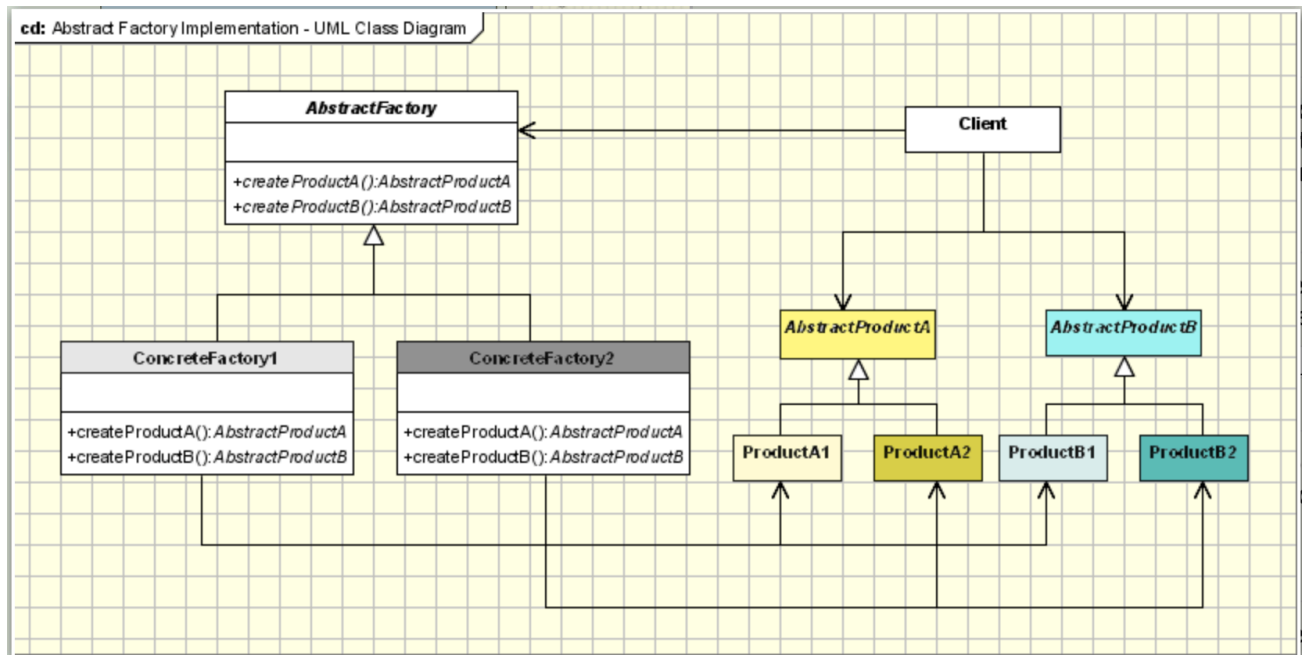
```
-----  
public class ConcreteProductB_White implements ProductB {  
    @Override  
    public void use() {  
    }  
}
```

```

public class Program {
    public static void main(String[] args) {
        AbstractFactory abstractFactory_Black = new ConcreteFactory_Black();
        ProductB productB_Black = abstractFactory_Black.createProductB();
        productB_Black.use();

        AbstractFactory abstractFactory_White = new ConcreteFactory_White();
        ProductA productA_White = abstractFactory_White.createProductA();
        productA_White.use();
    }
}

```



Builder – Usage

The Builder pattern separates **the construction of a complex object** from its **final representation**, allowing the same construction process to create different variations of an object by *passing in different parameters* or by *slightly altering construction steps*.

It is especially useful when objects require **multiple steps, optional parameters, or must be created in a specific order**.

When to Use Builder

Object construction is complex

The object has many fields, nested components, or requires multi-step initialization.

You want to avoid telescoping constructors

Instead of:

```
House h(4, true, false, true, 3, false, true); // unreadable
```

```
public interface Builder {  
    void buildPartA();  
  
    void buildPartB();  
  
    void buildPartC();  
  
    Product getProduct();  
}
```

```
-----  
public class ConcreteBuilderA implements Builder {  
  
    public ConcreteBuilderA() {  
        product = new Product();  
    }  
  
    @Override  
    public void buildPartA() {  
        product.attrib_1 = 1;  
    }  
  
    @Override  
    public void buildPartB() {  
        product.attrib_2 = 'A';  
    }  
}
```

```
@Override
public void buildPartC() {
    product.attrib_3 = 10.15;
}

@Override
public Product getProduct() {
    return product;
}

private Product product;
}
```

```
public class ConcreteBuilderB implements Builder {
    public ConcreteBuilderB() {
        product = new Product();
    }
```

```
@Override
public void buildPartA() {
    product.attrib_1 = 2;
}
```

```
@Override
public void buildPartB() {
    product.attrib_2 = 'B';
}
```

```
@Override
public void buildPartC() {
    product.attrib_3 = 20.25;
}
```

```
@Override
public Product getProduct() {
    return product;
}
```

```
private Product product;
}
```

```
public class Director {
    public void constructFullProduct(Builder builder) {
        builder.buildPartA();
        builder.buildPartB();
        builder.buildPartC();
    }

    public void constructSimpleProduct(Builder builder) {
        builder.buildPartA();
    }
}
```

```

public class Product {
    public void show() {

    }

    public void use() {

    }

    public Integer attrib_1;
    public char attrib_2;
    public double attrib_3;
}

```

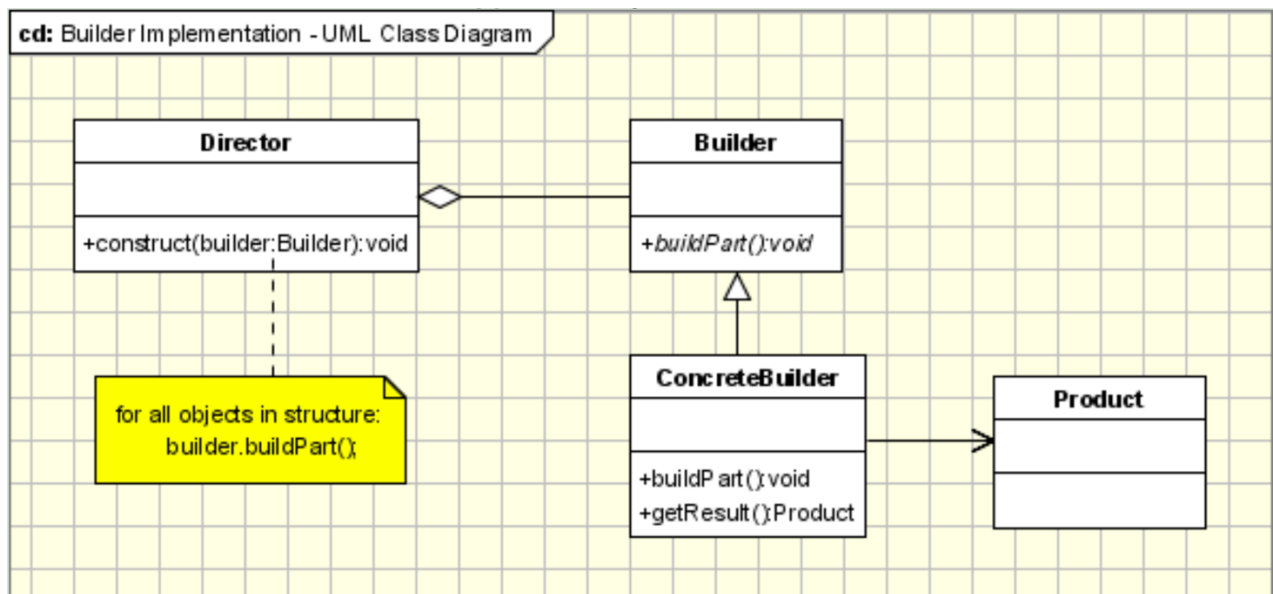
```

public class Program {
    public static void main(String[] args) {
        Director director = new Director();
        Builder builderA = new ConcreteBuilderA();

        director.constructFullProduct(builderA);
        builderA.getProduct().show();

        Builder builderB = new ConcreteBuilderB();
        director.constructFullProduct(builderB);
        builderB.getProduct().show();
    }
}

```



Prototype – Usage

The Prototype pattern is used to **create new objects by copying existing ones**, instead of instantiating them from scratch.

This is useful when object creation is **expensive, complex**, or when you want to **avoid large constructor logic**.

```
public interface Prototype {
```

```
    Prototype clone();
```

```
}
```

```
-----
```

```
public class ConcretePrototypeA implements Prototype {
```

```
    @Override
```

```
    public Prototype clone() {
```

```
        return new ConcretePrototypeA();
```

```
    }
```

```
}-----
```

```
public class ConcretePrototypeB implements Prototype {
```

```
    @Override
```

```
    public Prototype clone() {
```

```
        return new ConcretePrototypeB();
```

```
    }
```

```
}-----
```

```
public class Program {
```

```
    public static void main(String[] args) {
```

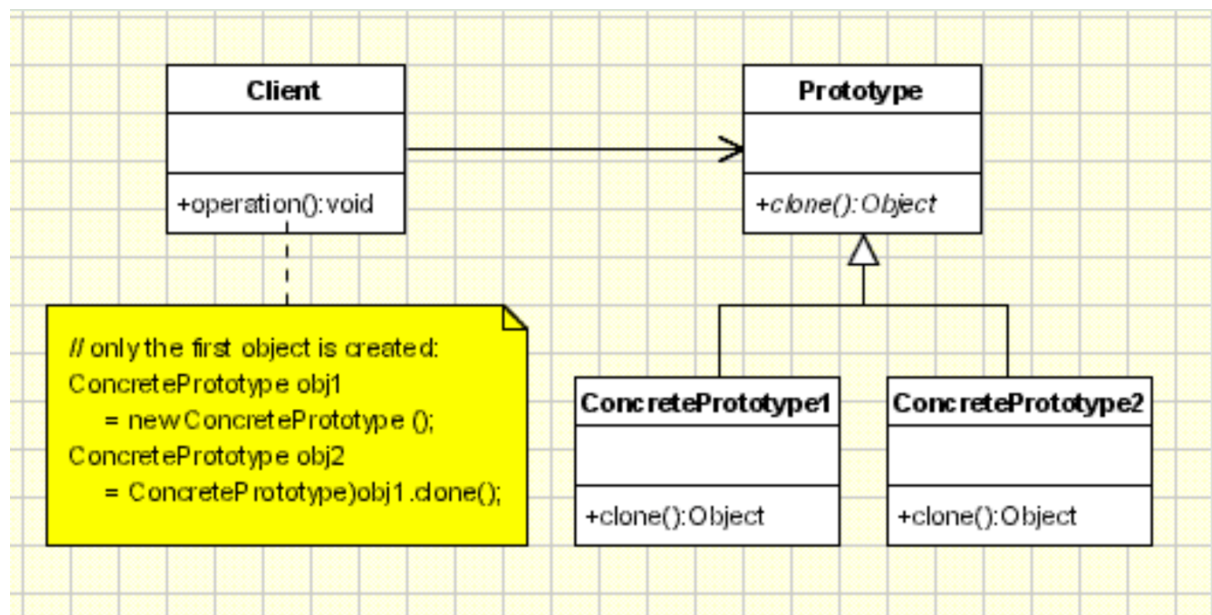
```
        Prototype prototype1 = new ConcretePrototypeA();
```

```
        Prototype prototype2 = prototype1.clone();
```

```
        prototype1 = prototype2.clone();
```

```
    }
```

```
}
```



Structural

Adapter – Usage

The Adapter pattern allows **incompatible interfaces** to work together.

It acts as a **translator** between a client expecting a `Target` interface and an existing class (`Adaptee`) that has a different interface.

The Adapter pattern is commonly used when you want to **reuse existing code** without modifying it.

When to Use Adapter

You want to integrate a class with an incompatible interface

The client expects methods that the existing class does not provide.

You want to reuse legacy or third-party code

Instead of modifying the original code, create an adapter to match the expected interface.

You want to decouple client code from the implementation

Clients depend only on the `Target` interface, not on the concrete `adaptee` class - we can change the underlying implementation (replace **Adaptee** with another class).

```
public class Adaptee {  
    public void adapteeOperation() {  
        System.out.println("Adaptee operation.");  
    }  
}
```

```
public class Adapter extends Target {  
  
    public Adapter(Adaptee adaptee) {  
  
        this.adaptee = adaptee;  
  
    }  
  
    @Override  
  
    public void targetOperation() {  
  
        adaptee.adapteeOperation();  
  
    }  
  
    private Adaptee adaptee;  
  
}
```

```

public class Target {

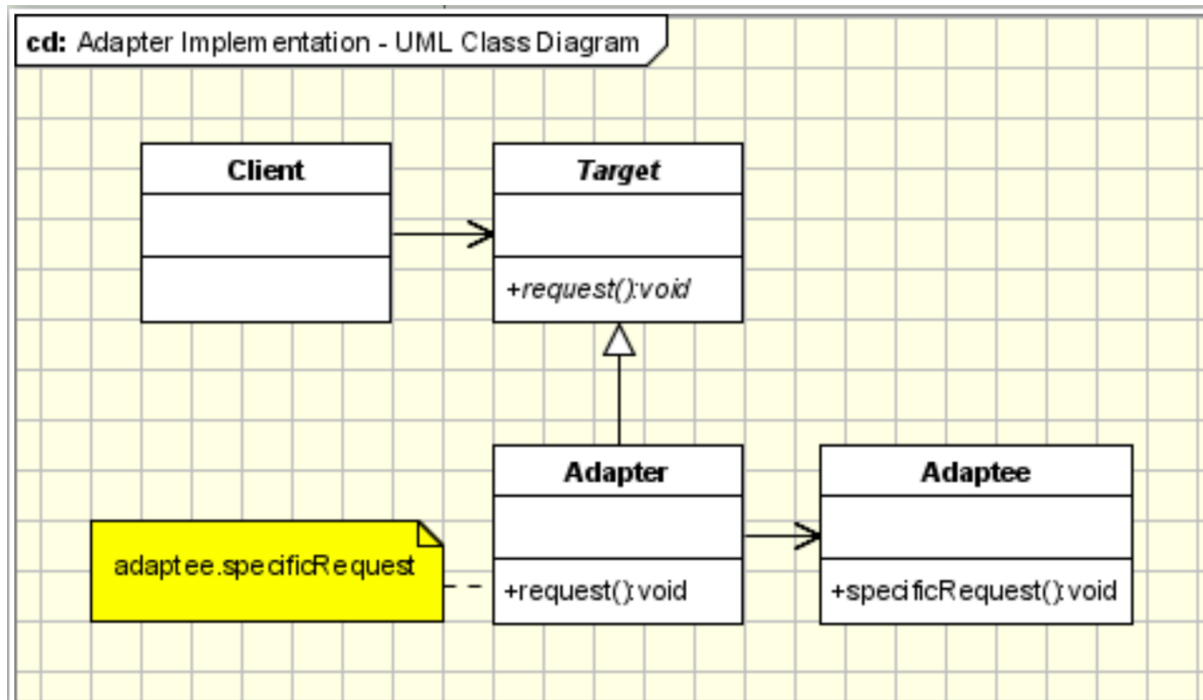
    public void targetOperation() {

        System.out.println("Target operation");

    }

}

```



Bridge – Usage

The **Bridge** design pattern is used when you want to **separate an abstraction from its implementation**, allowing both to vary independently. **The Abstraction delegates the work to the Implementation**. It is especially useful when:

You have multiple dimensions of variation

For example, different *shapes* (Circle, Square...) and different *rendering methods* (RasterRenderer, VectorRenderer).

The Bridge pattern prevents a combinatorial explosion of subclasses.

You want to avoid binding an implementation permanently

The abstraction can switch between implementations at runtime (e.g., switching renderers, platforms, APIs).

You want cleaner, testable, loosely-coupled code

Clients interact only with the abstraction, not the implementation details.
Implementations can be replaced, extended, or mocked easily.

You want to decouple high-level logic from low-level platform details

Useful in cross-platform applications:

- The abstraction contains business logic
- The implementation contains OS, database, graphics, or device-level operations

```
public class Abstraction {  
  
    public Abstraction(Implementation implementation) {  
  
        this.implementation = implementation;  
  
    }  
  
    public void operation() {  
  
        implementation.operationImplementation();  
  
    }  
  
    protected Implementation implementation;  
  
}
```

```
public class FinalAbstraction extends Abstraction {  
  
    public FinalAbstraction(Implementation implementation) {  
        super(implementation);  
    }  
  
    public void operation() {  
        System.out.println("Preprocessing...");  
        implementation.operationImplementation();  
        System.out.println("Postprocessing...");  
    }  
  
}
```

```
public interface Implementation {  
  
    public abstract void operationImplementation();  
  
}
```

```
public class ConcreteImplementationA implements Implementation {  
  
    @Override  
  
    public void operationImplementation() {  
  
        System.out.println("Operation implementation type A");  
  
    }  
  
}
```

```

public class ConcretImplementationB implements Implementation {

    @Override

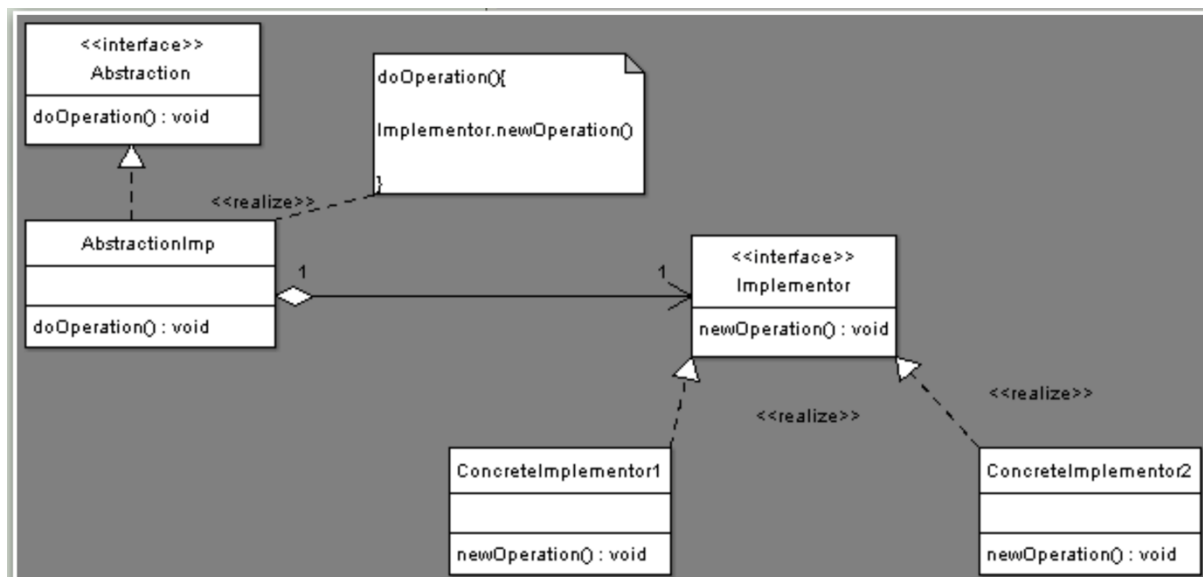
    public void operationImplementation() {

        System.out.println("Operation implementation type B");

    }

}

```



Composite – Usage

The **Composite** design pattern is used when you need to represent **hierarchical, tree-like structures** of objects, where **individual items (leaves)** and **groups of items (composites)** should be treated **uniformly**.

When to Use Composite

1. You want to represent part–whole hierarchies

Anything that forms a tree structure is a good candidate:

- Files and folders
 - UI components and containers
 - Game objects and groups
 - Organization charts
 - DOM trees
-

2. You want clients to treat individual objects and groups the same

Clients shouldn't need to check:

```
if (obj is leaf) { ... } else { ... }
```

```
public interface Component {
```

```
    void operation();
```

```
}
```

```
import java.util.ArrayList;

import java.util.List;

public class Composite implements Component {

    @Override

    public void operation() {

        for (Component component : components) {

            component.operation();

        }

    }

    public void addComponent(Component component) {

        components.add(component);

    }

    public void removeComponent(Component component) {

        components.remove(component);

    }

    public Component getComponent(int ind) {

        if (ind >= components.size() || components.size() < 0)

            return null;

        return components.get(ind);

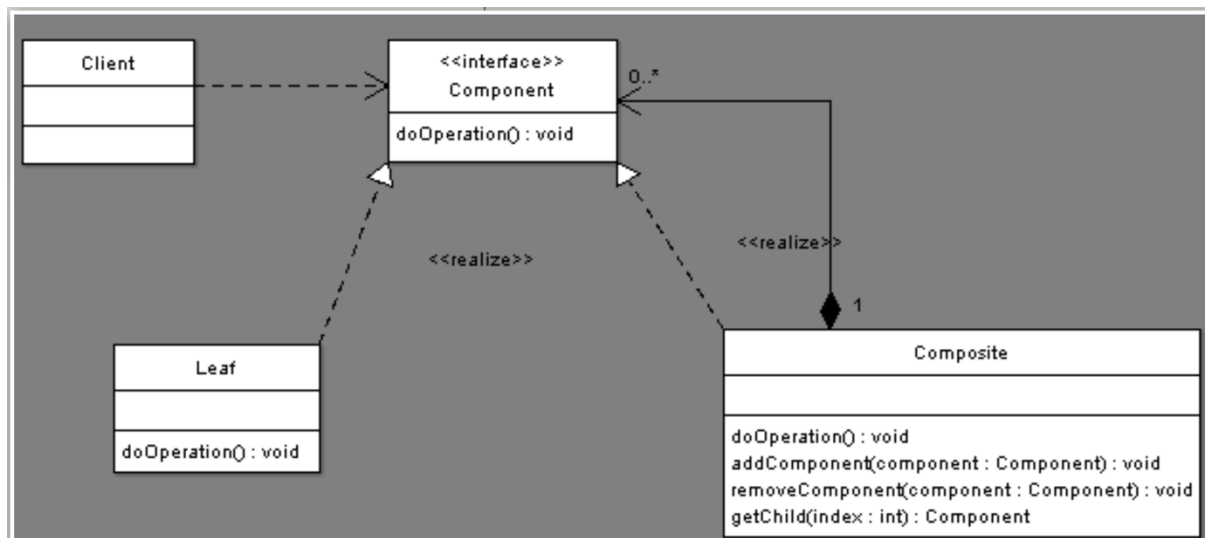
    }

    private List<Component> components = new ArrayList<Component>();

}
```

```
public class Leaf implements Component {  
  
    @Override  
  
    public void operation() {  
  
        System.out.println("Leaf operation implementation.");  
  
    }  
}
```

```
-----  
public class Program {  
  
    public static void main(String[] args) {  
  
        // Create leaf components  
  
        Component file1 = new Leaf();  
  
        Component file2 = new Leaf();  
  
        Component file3 = new Leaf();  
  
        // Create composite components (folders)  
  
        Composite folder1 = new Composite();  
  
        Composite folder2 = new Composite();  
  
        Composite rootFolder = new Composite();  
  
        // Build tree structure  
  
        folder1.addComponent(file1);  
  
        folder1.addComponent(file2);  
  
        folder2.addComponent(file3);  
  
        rootFolder.addComponent(folder1);  
  
        rootFolder.addComponent(folder2)  
  
        // Display structure  
  
        rootFolder.operation();  
  
    }  
}
```

Decorator – Usage

The **Decorator** design pattern allows you to **dynamically add behavior** to objects **without modifying their class**.

It is especially useful when you want to add **responsibilities at runtime** in a flexible and reusable way.

When to Use Decorator

1. You want to add behavior without subclassing

- Instead of creating multiple subclasses for every combination of features, decorators can wrap an object and add features on demand.

2. You want to extend functionality dynamically

- Features can be stacked: wrap an object with multiple decorators to combine behaviors.

3. You want to follow the Open/Closed Principle

- Classes remain closed for modification, open for extension.
- New behavior is added by creating a new decorator class, not by changing existing code.

```
public interface Component {
```

```
    void operation();
```

```
}
```

```
-----  
public class ConcreteComponent implements Component {
```

```
    @Override
```

```
    public void operation() {
```

```
        System.out.println("Component operation implementation.");
```

```
    }
```

```
}
```

```
-----
```

```
public abstract class Decorator implements Component {
```

```
    protected final Component component;
```

```
    public Decorator(Component component) {
```

```
        this.component = component;
```

```
    }
```

```
}-----
```

```
public class ConcreteDecoratorExtendingFunctionality extends Decorator {
```

```
    public ConcreteDecoratorExtendingFunctionality(Component component) {
```

```
        super(component);
```

```
    }
```

```
    @Override
```

```
    public void operation() {
```

```
        component.operation();
```

```
        addedOperation();
```

```
    }
```

```
    private void addedOperation() {
```

```
        System.out.println("Decorator adds functionality!");
```

```
    }
```

```
}-----
```

```

public class ConcreteDecoratorExtendingState extends Decorator {

    private final State state;

    public ConcreteDecoratorExtendingState(Component component, State state) {

        super(component);

        this.state = state;

    }

    @Override

    public void operation() {

        component.operation();

        System.out.println("State (only x and y): " + state.x + " " + state.y);

    }

}

```

```

public class Program {

    public static void main(String[] args) {

        // Base component

        Component component = new ConcreteComponent();

        System.out.println("--- Plain component ---");

        component.operation();

        // Add extra functionality via decorator

        Component withFunctionality = new ConcreteDecoratorExtendingFunctionality(component);

        System.out.println("\n--- Component with extra functionality ---");

        withFunctionality.operation();

        // Add state-extending decorator

        State state = new State(1, 2, 3, 'a', 'b', 'c');

        Component withState = new ConcreteDecoratorExtendingState(withFunctionality, state);
    }

}

```

```

System.out.println("\n--- Component with functionality and state ---");

withState.operation();

// Nest another functionality decorator on top of state

Component nested = new ConcreteDecoratorExtendingFunctionality(withState);

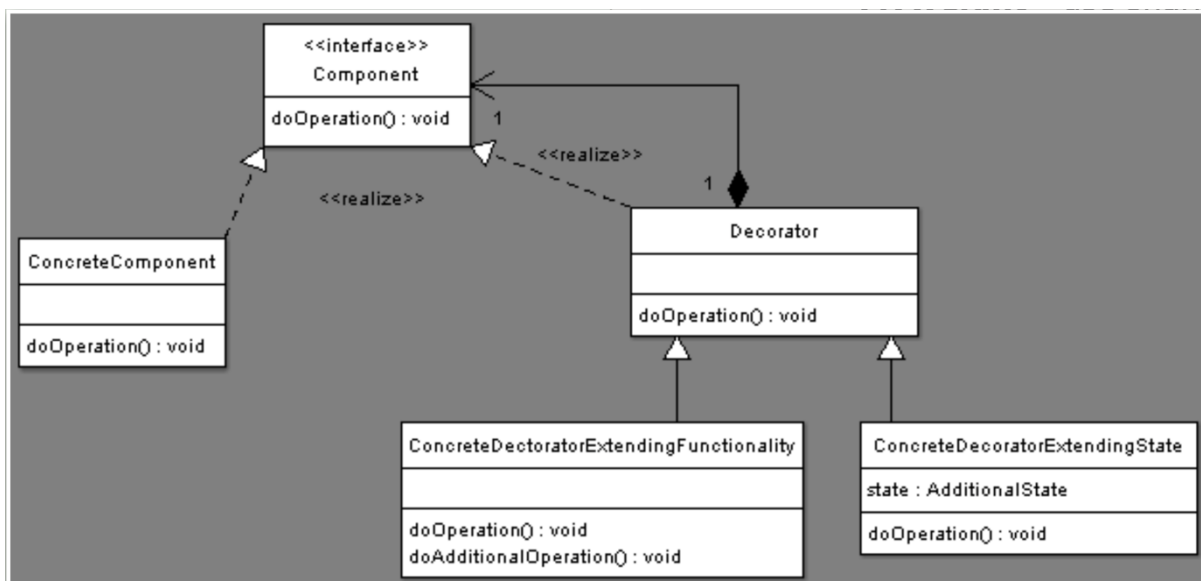
System.out.println("\n--- Nested decorators (state + extra functionality) ---");

nested.operation();

}

}

```



Facade – Usage

When to Use

Use the **Facade** pattern when you want to:

- Provide a *simple, unified interface* to a **complex subsystem**.
- Reduce coupling between client code and many subsystem classes.
- Hide messy initialization, configuration, or sequencing logic behind one clean API.
- Make existing code easier to use without modifying the subsystem itself.

What It Solves

Without a facade, the client must understand:

- which subsystem classes to call,
- in what order,
- with which parameters,
- and what depends on what.

The Facade wraps all that complexity and exposes a **minimal, friendly interface**.

Key Idea

The facade *does not replace* subsystem classes — it simply **delegates** to them internally. Clients talk only to the facade; advanced users may still use subsystem classes directly.

```
public class HomeTheaterFacade {  
  
    private final AudioSystem audioSystem;  
  
    private final VideoSystem videoSystem;  
  
    private final StreamingService streamingService;  
  
    public HomeTheaterFacade(AudioSystem audioSystem, VideoSystem videoSystem, StreamingService  
streamingService) {  
  
        this.audioSystem = audioSystem;  
  
        this.videoSystem = videoSystem;  
  
        this.streamingService = streamingService;  
  
    }  
}
```

```
public void watchMovie(String user, String movieTitle) {  
  
    System.out.println("HomeTheaterFacade: preparing to watch a movie...");  
  
    audioSystem.on();  
  
    audioSystem.setVolume(15);  
  
    videoSystem.on();  
  
    videoSystem.setInput("Streaming");  
  
    streamingService.connect(user);  
  
    streamingService.play(movieTitle);  
  
    System.out.println("HomeTheaterFacade: enjoy the movie!");  
  
}  
  
public void endMovie() {  
  
    System.out.println("HomeTheaterFacade: shutting movie down...");  
  
    streamingService.disconnect();  
  
    videoSystem.off();  
  
    audioSystem.off();  
  
    System.out.println("HomeTheaterFacade: system is off.");  
  
}  
}
```

```
public class Program {  
  
    public static void main(String[] args) {  
  
        AudioSystem audioSystem = new AudioSystem();  
  
        VideoSystem videoSystem = new VideoSystem();  
  
        StreamingService streamingService = new StreamingService();  
  
  
        HomeTheaterFacade facade = new HomeTheaterFacade(audioSystem, videoSystem,  
streamingService);  
  
        facade.watchMovie("john.doe", "The Facade Awakens");  
  
        System.out.println();  
  
        facade.endMovie();  
  
    }  
}
```

Flyweight – Usage

The **Flyweight** design pattern is used to **minimize memory usage** by sharing as much data as possible between similar objects.

It is especially useful when dealing with **a large number of fine-grained objects**.

Key Concepts

Intrinsic state

- Shared among multiple flyweight objects.
- Stored **inside the flyweight**.
- Example: the **glyph shape** of a character, or the **texture** of a tree in a game.

Extrinsic state

- Not shared; varies per usage.
 - Supplied **externally** by the client at runtime.
 - Example: the **position** of a tree on the map, or the **location** of a character in a document.
-

When to Use Flyweight

1. You need to create **many similar objects** and want to reduce memory usage.
2. Part of the object's state can be **shared** (intrinsic), while the rest is **context-specific** (extrinsic).
3. You want to **centralize shared data** and allow **runtime customization** via extrinsic state.

```
public interface IFlyweight {  
  
    void operation(ExtrinsicState state);  
  
}
```

```
public class ConcreteFlyweight implements IFlyweight {

    private final IntrinsicState intrinsic;

    public ConcreteFlyweight(String texture, String color) {

        this.intrinsic = new IntrinsicState(texture, color);

    }

    @Override

    public void operation(ExtrinsicState extrinsic) {

        render(extrinsic, intrinsic);

    }

    private void render(ExtrinsicState extrinsic, IntrinsicState intrinsic) {

        System.out.printf(

            "Rendering object at (%d, %d) with texture '%s' and color '%s'%n",

            extrinsic.getX(), extrinsic.getY(), intrinsic.getTexture(), intrinsic.getColor());

    }

}
```

```
public class ExtrinsicState {  
  
    private final int x;  
  
    private final int y;  
  
    public ExtrinsicState(int x, int y) {  
  
        this.x = x;  
  
        this.y = y;  
  
    }  
  
    public int getX() {  
  
        return x;  
  
    }  
  
    public int getY() {  
  
        return y;  
  
    }  
  
}
```

```
public class IntrinsicState {  
  
    private final String texture;  
  
    private final String color;  
  
    public IntrinsicState(String texture, String color) {  
  
        this.texture = texture;  
  
        this.color = color;  
  
    }  
  
    public String getTexture() {  
  
        return texture;  
  
    }  
  
    public String getColor() {  
  
        return color;  
  
    }  
}
```

```

public class Program {

    public static void main(String[] args) {

        FlyweightFactory factory = new FlyweightFactory();

        ExtrinsicState state1 = new ExtrinsicState(10, 20);

        ExtrinsicState state2 = new ExtrinsicState(50, 80);

        ConcreteFlyweight tree1 = factory.getFlyweight("Oak", "Green");

        ConcreteFlyweight tree2 = factory.getFlyweight("Oak", "Green");

        tree1.operation(state1);

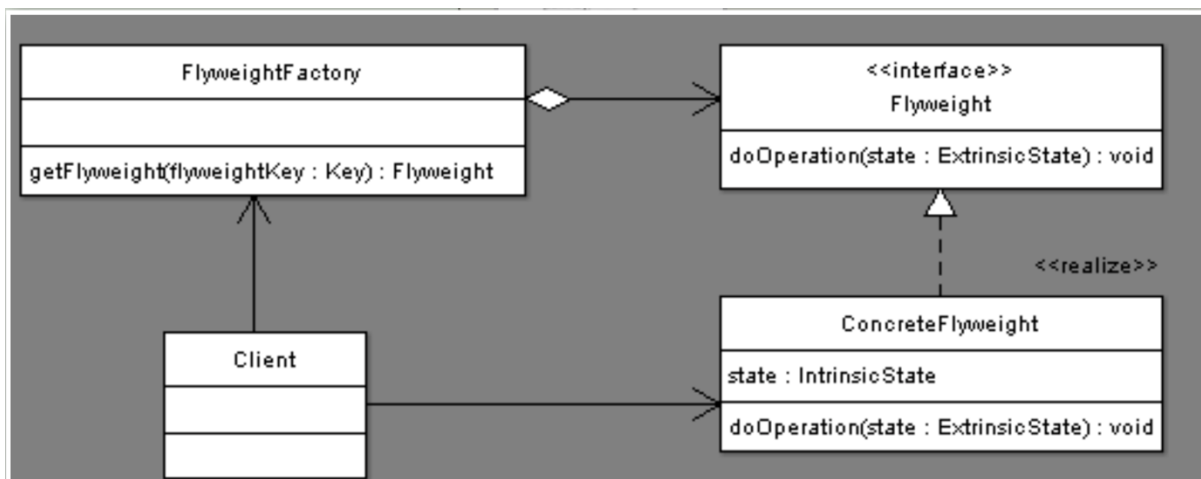
        tree2.operation(state2);

        System.out.println("Flyweights created: " + factory.getFlyweightCount());

    }

}

```



Memento – Usage

The **Memento** design pattern is used to capture and restore an object's **internal state** without violating **encapsulation**.
It lets you implement **undo / rollback**, snapshots, or state history in a clean and controlled way.

Intent

- Save an object's state (**snapshot**) at a specific moment.
- Restore the state later (undo).
- Keep internal details hidden from the outside world.

The pattern involves three roles:

1. Originator

The object whose state you want to save.

- Creates Mementos.
- Restores its state from a Memento.

2. Memento

Stores the **internal state** of the Originator.

- Immutable.
- No one except Originator should modify or inspect it.

3. Caretaker

Manages saving and restoring snapshots.

- Stores mementos.
- Never inspects their contents.

When to Use Memento

Use the Memento pattern when:

- You need **undo / redo** (text editors, GUIs, games, command history).
- You want to implement **save/load** functionality.
- You want to **encapsulate state history** without exposing object internals.
- You need to support **rollback** to a previous valid state.

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public class CareTaker {
```

```
    private final List<Memento> mementos = new ArrayList<>();
```

```
    public void add(Memento memento) {
```

```
        mementos.add(memento);
```

```
    }
```

```
    public Memento get(int index) {
```

```
        return mementos.get(index);
```

```
    }
```

```
}
```

```
public class Memento {
```

```
    private final String state;
```

```
    public Memento(String state) {
```

```
        this.state = state;
```

```
    }
```

```
    public String getState() {
```

```
        return state;
```

```
    }
```

```
}
```

```
public class Originator {  
  
    private String state;  
  
    public void setState(String state) {  
  
        this.state = state;  
  
        System.out.println("Originator: state changed to \"" + state + "\"");  
  
    }  
  
    public String getState() {  
  
        return state;  
  
    }  
  
    public Memento saveToMemento() {  
  
        System.out.println("Originator: saving state to Memento.");  
  
        return new Memento(state);  
  
    }  
  
    public void restoreFromMemento(Memento memento) {  
  
        this.state = memento.getState();  
  
        System.out.println("Originator: state restored to \"" + state + "\"");  
  
    }  
  
}
```

```

public class Program {

    public static void main(String[] args) {

        Originator originator = new Originator();

        CareTaker careTaker = new CareTaker();

        originator.setState("State #1");

        originator.setState("State #2");

        careTaker.add(originator.saveToMemento());

        originator.setState("State #3");

        careTaker.add(originator.saveToMemento());

        originator.setState("State #4");

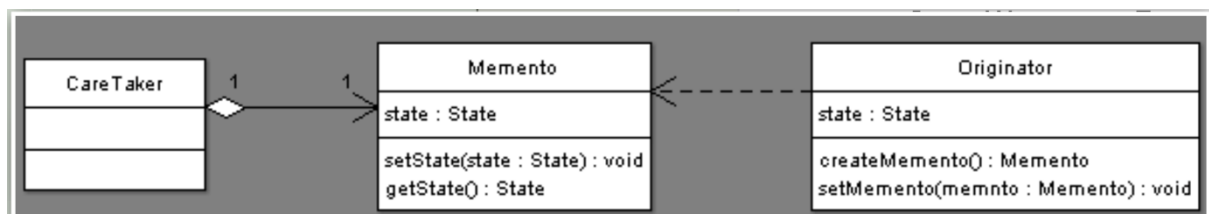
        originator.restoreFromMemento(careTaker.get(0));

        originator.restoreFromMemento(careTaker.get(1));

    }

}

```



Proxy – Usage

The **Proxy** design pattern provides a **surrogate** or **placeholder** object that controls access to another object (the *real subject*).

You use a Proxy when you **want to add extra behavior before or after accessing a real object**—without modifying the real object itself.

✓ When to Use the Proxy Pattern

1. Access Control

Control who can call methods on an object.

Example:

A `SecureProxy` checks login credentials before allowing access to a `BankAccount`.

2. Lazy Initialization (Virtual Proxy)

Delay creating a heavy object until it is actually needed.

Example:

Loading a big image file only when `.display()` is called.

3. Logging, Monitoring, or Statistics

Record when a method is used, without changing the real object.

Example:

A `LoggingProxy` prints every request to a database.

4. Remote Proxy (RPC, networking)

The real object lives on a different machine, and the proxy handles communication.

Example:

Calling a method on a server through a stub proxy.

5. Protection Proxy

Different behavior depending on user permissions.

Example:

Admin users can edit records; regular users can only view them.

```
public class Proxy implements Subject {

    private RealSubject realSubject;

    private boolean accessGranted;

    public void setAccess(boolean granted) {

        this.accessGranted = granted;

    }

    @Override

    public void request(String payload) {

        if (!checkAccess()) {

            System.out.println("Proxy: access denied. Cannot forward request.");

            return;

        }

        if (realSubject == null) {

            System.out.println("Proxy: creating RealSubject on demand.");

            realSubject = new RealSubject();

        }

        logAccess(payload);

        realSubject.request(payload);

    }

    private boolean checkAccess() {

        System.out.println("Proxy: checking access...");

        return accessGranted;

    }

    private void logAccess(String payload) {

        System.out.println("Proxy: logging access for payload: " + payload);

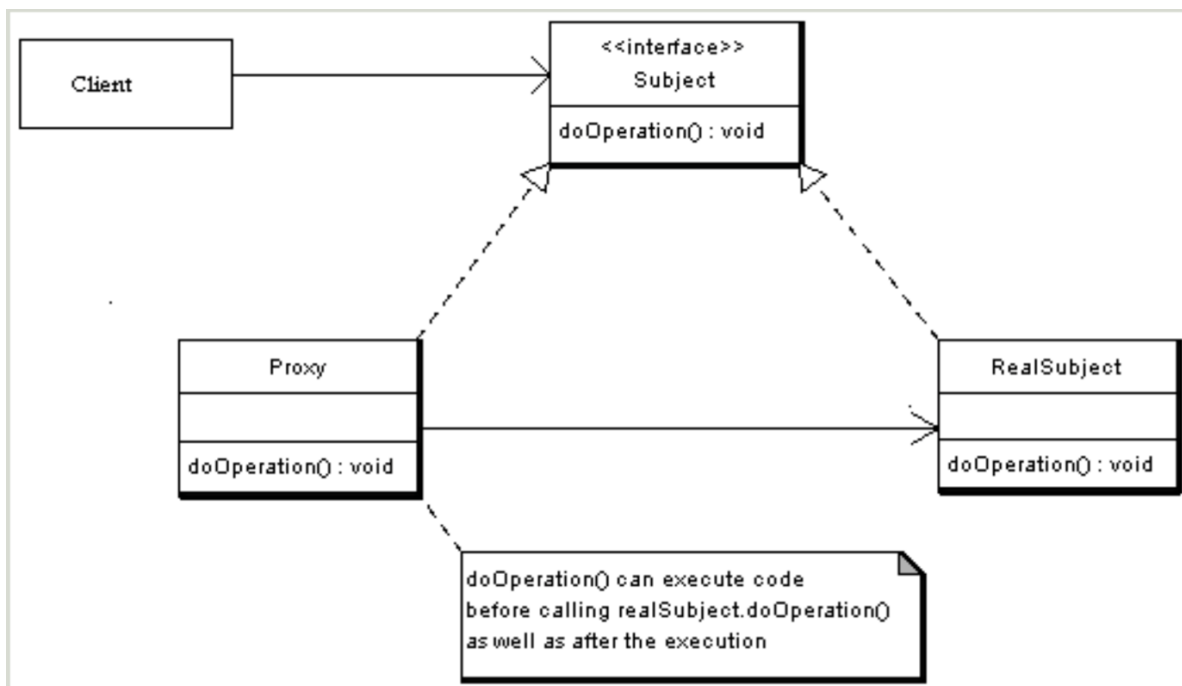
    }

}
```

```
public interface Subject {  
  
    void request(String payload);  
  
}
```

```
-----  
  
public class RealSubject implements Subject {  
  
    @Override  
  
    public void request(String payload) {  
  
        System.out.println("RealSubject: handling request with payload: " + payload);  
  
    }  
  
}
```

```
-----  
  
public class Program {  
  
    public static void main(String[] args) {  
  
        Proxy proxy = new Proxy();  
  
        proxy.setAccess(false);  
  
        proxy.request("First call");  
  
        proxy.setAccess(true);  
  
        proxy.request("Second call");  
  
    }  
  
}
```



Behavioral

Command – Usage

The **Command** design pattern turns a request into a **standalone object**.

This allows you to parameterize methods with commands, queue them, store them, undo them, or log them.

It decouples:

- **The object that invokes an operation** (Invoker)
 - **The object that actually performs the operation** (Receiver)
-

Intent

- Encapsulate a request as an object
 - Support **undo/redo**, logging, command queues, and macros
 - Decouple *what is done* from *how it is done*
-

When to Use the Command Pattern

1. Undo/Redo functionality

Store previous states or inverse commands.

Example:

Text editor undo stack (typing, deleting, formatting).

2. Command history / logging

Keep a history of executed actions.

Example:

Storing executed database queries.

3. Queued or scheduled commands

Execute commands later or in order.

Example:

Task scheduler, game engine event queue.

4. GUI buttons / menu items

Each button can map to a command object.

Example:
"Copy", "Paste", "Save", "Delete" as commands.

5. Macro commands

Combine multiple commands into one.

Example:
"Build & Run" in IDE (runs multiple steps).

Roles in the Pattern

1. Command (Interface)

Declares the `execute()` method.

2. ConcreteCommand

Implements the command and stores a reference to the Receiver.

3. Receiver

Knows how to perform the actual work.

4. Invoker

Triggers command execution.

5. Client

Creates the command objects and assigns them to the invoker.

```
public interface Command {  
  
    void execute();  
  
}
```

```
public class ConcreteCommand implements Command {  
  
    public ConcreteCommand(Receiver receiver) {  
  
        this.receiver = receiver;  
  
    }  
  
    @Override  
  
    public void execute() {  
  
        receiver.action();  
  
    }  
  
    private Receiver receiver;  
  
}
```

```
public class Invoker {  
  
    public void setCommand(Command command) {  
  
        this.command = command;  
  
    }  
  
    public void executeCommand() {  
  
        if (command != null)  
  
            command.execute();  
  
    }  
  
    private Command command;  
  
}
```

```
public class Receiver {  
  
    public void action() {  
  
        System.out.println("Actual action");  
  
    }  
  
}
```



```

public class Program {

    public static void main(String[] args) {

        Receiver receiver = new Receiver();

        Command command = new ConcreteCommand(receiver);

        Invoker invoker = new Invoker();

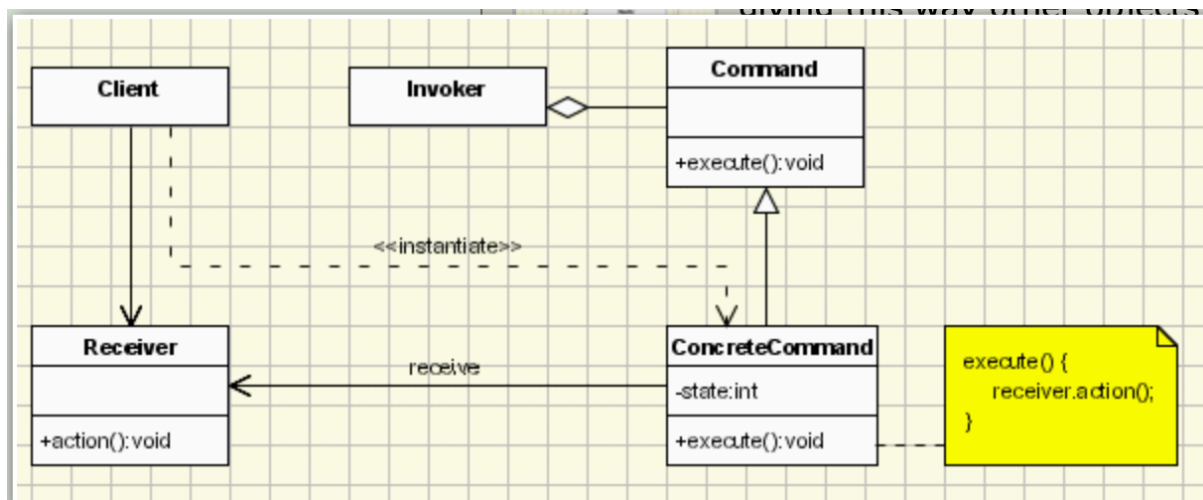
        invoker.setCommand(command);

        invoker.executeCommand();

    }

}

```



Mediator – Usage

The **Mediator** pattern centralizes communication between multiple objects. Instead of objects calling each other directly, they interact **only through a mediator**, which reduces coupling and simplifies collaboration logic.

When to Use the Mediator Pattern

1. Many objects need to communicate

When interactions between objects become complex and tightly coupled, Mediator organizes and centralizes this communication.

2. You want to reduce dependencies

Colleague objects do **not** reference each other. They depend only on the mediator, making the system more modular and easier to maintain.

3. You want a single place for interaction logic

Rules for how objects should interact are stored in one place (the mediator), instead of being spread across many classes.

4. You want to easily extend or modify interactions

Changing the mediator changes system behavior **without modifying the individual components**.

Example Scenario

Chat room system

- Each user sends messages *to the mediator* (chat room)
- Chat room forwards messages to all other users
- Users remain simple, and do not depend on each other

This avoids a fully connected network of users referencing every other user.

Benefits

- **Loose coupling** between components
- **Centralized logic** for communication
- **Better maintainability** and organization
- **Easier scalability** (adding new components requires no changes to others)

Drawbacks

- Mediator can grow large and complex (“god object”)
- Moves complexity to one place (but still more manageable)

```
interface Mediator {
```

```
    void notify(Colleague sender, String event);
```

```
}
```

```

class ConcreteMediator implements Mediator {

    private ConcreteColleague1 colleague1;

    private ConcreteColleague2 colleague2;

    public void setColleague1(ConcreteColleague1 colleague1) {

        this.colleague1 = colleague1;

    }

    public void setColleague2(ConcreteColleague2 colleague2) {

        this.colleague2 = colleague2;

    }

    @Override

    public void notify(Colleague sender, String event) {

        if ("A".equals(event)) {

            System.out.println("Mediator reacts to A and triggers:");

            if (colleague2 != null) {

                colleague2.doC();

            }

        } else if ("D".equals(event)) {

            System.out.println("Mediator reacts to D and triggers:");

            if (colleague1 != null) {

                colleague1.doB();

            }

        }

    }

}

```

```

abstract class Colleague {

    protected Mediator mediator;

    public Colleague(Mediator mediator) {

        this.mediator = mediator;

    }

    public void setMediator(Mediator mediator) {

        this.mediator = mediator;

    }

}

-----
class ConcreteColleague1 extends Colleague {

    public ConcreteColleague1(Mediator mediator) {

        super(mediator);

    }

    public void doA() {

        System.out.println("Colleague1: Doing A");

        if (mediator != null) {

            mediator.notify(this, "A");

        }

    }

    public void doB() {

        System.out.println("Colleague1: Doing B");

        if (mediator != null) {

            mediator.notify(this, "B");

        }

    }

}

```

```
class ConcreteColleague2 extends Colleague {  
  
    public ConcreteColleague2(Mediator mediator) {  
  
        super(mediator);  
  
    }  
  
    public void doC() {  
  
        System.out.println("Colleague2: Doing C");  
  
        if (mediator != null) {  
  
            mediator.notify(this, "C");  
  
        }  
    }  
  
    public void doD() {  
  
        System.out.println("Colleague2: Doing D");  
  
        if (mediator != null) {  
  
            mediator.notify(this, "D");  
  
        }  
    }  
}
```

```

public class Program {

    public static void main(String[] args) {

        ConcreteMediator mediator = new ConcreteMediator();

        ConcreteColleague1 c1 = new ConcreteColleague1(mediator);

        ConcreteColleague2 c2 = new ConcreteColleague2(mediator);

        mediator.setColleague1(c1);

        mediator.setColleague2(c2);

        System.out.println("Client triggers operation A:");

        c1.doA();

        System.out.println();

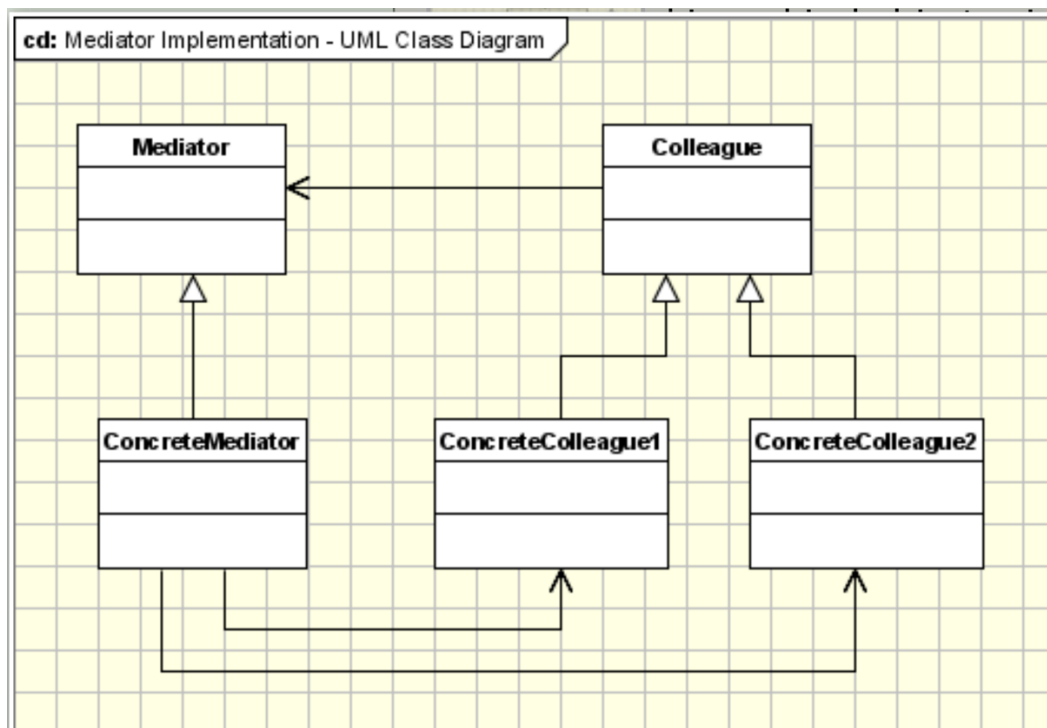
        System.out.println("Client triggers operation D:");

        c2.doD();

    }

}

```



Observer – Usage

The **Observer** pattern defines a one-to-many dependency: when one object (**Subject**) changes its state, all dependent objects (**Observers**) are automatically notified.

It is useful when you want to maintain consistency between related objects without tightly coupling them.

When to Use the Observer Pattern

1. One object should notify many others

Whenever a single state change must update multiple components (UI panels, logs, dependent objects, etc.).

2. You want loose coupling

Observers do not need to know details about the subject.
The subject does not need to know who the observers are.

3. You need dynamic subscription / unsubscription

Observers can be added or removed at runtime.

4. You want a reactive, event-driven system

The Observer pattern is the basis for:

- Event listeners
 - UI frameworks
 - Reactive programming
 - Messaging systems
-

Example Use Cases

- **GUI systems** → buttons notify listeners when clicked
 - **Game development** → entities react to health changes, environment events
 - **Stock market app** → stock data service notifies charts, logs, alerts
 - **File watcher** → any file change notifies tools/services
-

Benefits

- **Loose coupling** between subject and observers

- **Flexible:** observers can be added/removed dynamically
 - **Open for extension:** adding new observers requires no subject changes
-

Drawbacks

- Observers may cause unexpected updates or chain reactions
- Debugging can be harder (indirect communication)
- If many observers exist → performance overhead

```
public interface Observer {
```

```
    void update(Observable source, int state);
```

```
}
```

```
-----  
public class ConcreteObserverA implements Observer {
```

```
    @Override
```

```
    public void update(Observable source, int state) {
```

```
        System.out.println("Observer A notified. New state = " + state);
```

```
    }
```

```
}
```

```
-----  
public class ConcreteObserverB implements Observer {
```

```
    @Override
```

```
    public void update(Observable source, int state) {
```

```
        System.out.println("Observer B notified. New state = " + state);
```

```
    }
```

```
}
```

```
public class Observable {  
  
    protected int state = 0;  
  
    private final List<Observer> observers = new ArrayList<>();  
  
    public void attach(Observer observer) {  
  
        observers.add(observer);  
  
    }  
  
    public void detach(Observer observer) {  
  
        observers.remove(observer);  
  
    }  
  
    protected void notifyObservers() {  
  
        for (Observer observer : observers) {  
  
            observer.update(this, state);  
  
        }  
  
    }  
}
```

```
public class ConcreteObservable extends Observable {  
  
    public void setState(int newState) {  
  
        this.state = newState;  
  
        notifyObservers();  
  
    }  
  
    public int getState() {  
  
        return state;  
  
    }  
}
```

```

public class Program {

    public static void main(String[] args) {

        ConcreteObservable observable = new ConcreteObservable();

        ConcreteObserverA observerA = new ConcreteObserverA();

        ConcreteObserverB observerB = new ConcreteObserverB();

        observable.attach(observerA);

        observable.attach(observerB);

        System.out.println("Setting state to 1");

        observable.setState(1);

        System.out.println("Setting state to 2");

        observable.setState(2);

        observable.detach(observerA);

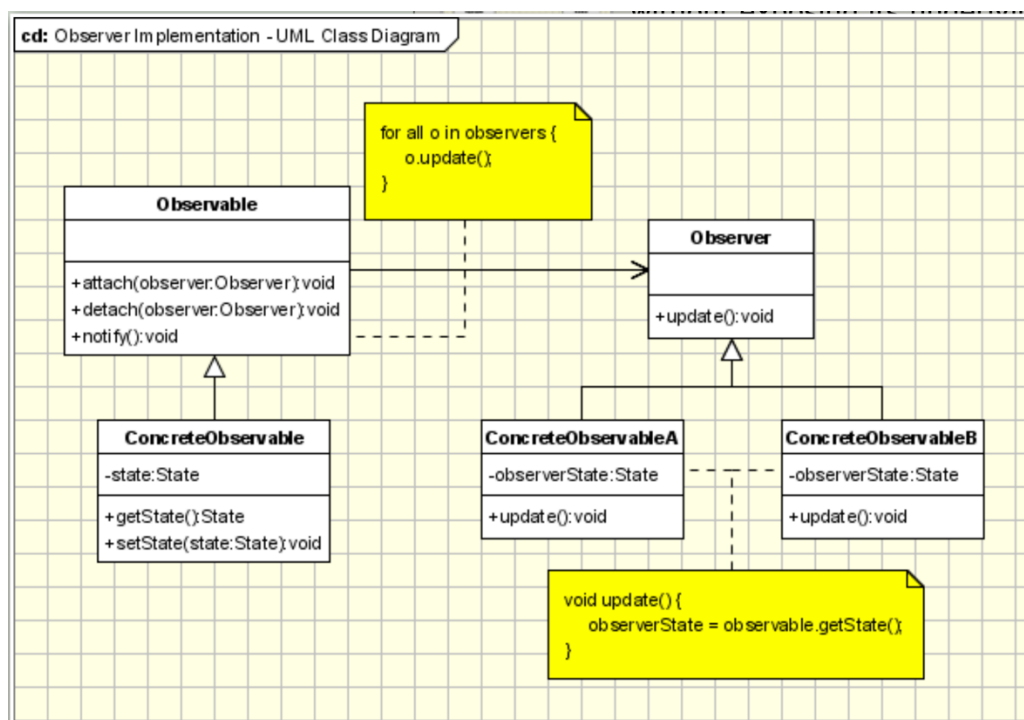
        System.out.println("Detached Observer A, setting state to 3");

        observable.setState(3);

    }

}

```



Visitor – Usage

The **Visitor** pattern allows you to separate an operation from the object structure on which it operates. It lets you add new behaviors *without modifying* the element classes by introducing a new "visitor" class for each behavior.

It is especially useful when you have a complex object structure and want to apply different operations to its elements over time.

When to Use the Visitor Pattern

1. You need to add new operations frequently

Instead of changing existing classes, you simply create a new Visitor.

2. You work with a stable object structure

The elements do not change often, but their behaviors might.

Examples:

- AST nodes in compilers
- XML/JSON processing
- File systems (files & folders)
- Scene graphs in rendering systems

3. You want to follow the Open/Closed Principle

Add new behaviors without modifying existing classes.

4. You need double dispatch

The visitor chooses the correct method based on:

- the visitor type
- the element type

This solves the issue where a single virtual call is not enough.

Benefits

- **Easy to add new operations** (just add a new Visitor)
- **Separates algorithms from data structures**
- Encourages **single responsibility**
- Can accumulate results across the whole structure (e.g., count nodes, compute size)

Drawbacks

- Harder to add new element types (must update all visitors)
 - Slightly more complex design with double dispatch
 - Tight coupling between elements and the Visitor interface
-

Example Use Cases

- Compiler design: type checking, optimizing, code generation
- File system traversal: computing sizes, printing structure
- UI frameworks: rendering, event propagation
- Data exporters: JSON → XML → YAML conversions

```
public interface Visitor {
```

```
    void visit(ConcreteElementA element);
```

```
    void visit(ConcreteElementB element);
```

```
}
```

```
public class ConcreteVisitor1 implements Visitor {
```

```
    @Override
```

```
    public void visit(ConcreteElementA element) {
```

```
        System.out.println("ConcreteVisitor1 visiting ElementA: " + element.getDataA());
```

```
    }
```

```
    @Override
```

```
    public void visit(ConcreteElementB element) {
```

```
        System.out.println("ConcreteVisitor1 visiting ElementB: " + element.getDataB());
```

```
    }
```

```
}
```

```
public class ConcreteVisitor2 implements Visitor {

    @Override

    public void visit(ConcreteElementA element) {

        System.out.println( "ConcreteVisitor2 processing ElementA differently: " +
element.getDataA().length() + " characters");

    }

    @Override

    public void visit(ConcreteElementB element) {

        System.out.println("ConcreteVisitor2 processing ElementB differently: " + (element.getDataB() * 2));

    }

}
```

```
public interface Element {

    void accept(Visitor visitor);

}
```

```
public class ConcreteElementA implements Element {

    @Override

    public void accept(Visitor visitor) {

        visitor.visit(this);

    }

    public String getDataA() {

        return "Data from Element A";

    }

}
```

```
public class ConcreteElementB implements Element {  
  
    @Override  
  
    public void accept(Visitor visitor) {  
  
        visitor.visit(this);  
  
    }  
  
    public int getDataB() {  
  
        return 42;  
  
    }  
}
```

```
public class Program {  
  
    public static void main(String[] args) {  
  
        ConcreteElementA elementA = new ConcreteElementA();  
  
        ConcreteElementB elementB = new ConcreteElementB();  
  
  
        ConcreteVisitor1 visitor1 = new ConcreteVisitor1();  
  
        ConcreteVisitor2 visitor2 = new ConcreteVisitor2();  
  
  
        System.out.println("=== Visitor 1 ===");  
  
        elementA.accept(visitor1);  
  
        elementB.accept(visitor1);  
  
  
        System.out.println("\n=== Visitor 2 ===");  
  
        elementA.accept(visitor2);  
  
        elementB.accept(visitor2);  
  
    }  
}
```

cd: Visitor Implementation - UML Class Diagram

